

Week 8

System Protection

IN 1011 Operating Systems

Systems Protection

- Goals and Principles of Protection
- Protection through Domains
- Access Matrix
- Revocation of Access Rights
- Access Control Policies

Systems Protection

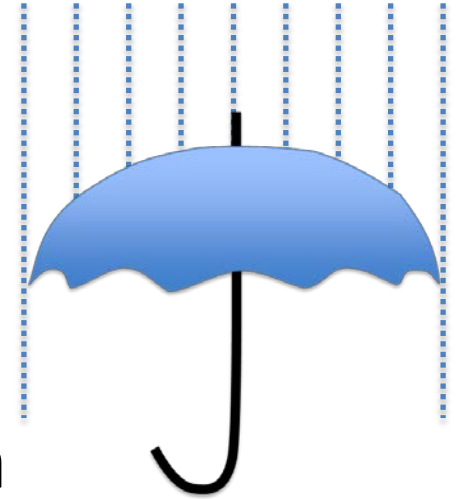
- Goals and Principles of Protection
- Protection through Domains
- Access Matrix
- Revocation of Access Rights
- Access Control Policies

Goals of Protection

- A computer consists of a collection of objects, hardware or software
- Each object has a unique name and can be accessed through a well-defined set of operations
- Protection problem - *ensure that each object is accessed correctly and only by those processes that are allowed to do so*

Goals of Protection

- Protection involves **controlling the access of processes and users** to the resources defined by a computer system
- The processes in an operating system must be **protected from one another's** activities.

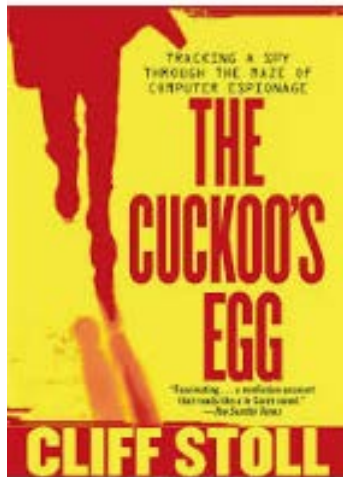


Goals of Protection

- To provide this protection, we can use various mechanisms to ensure that only processes that have gained proper **authorisation** from the operating system can operate on the files, memory segments, CPU, networking, and other resources of a system.
- These mechanisms must provide a means for **specifying** the controls to be imposed, together with a **means of enforcement**

Goals of Protection

- Protection was originally conceived as an adjunct to multiprogramming operating systems, so that **untrustworthy users might safely share** a common logical name space, such as a directory of files, or a common physical name space, such as memory.



Modern protection concepts have evolved to increase the reliability of any complex system that makes use of shared resources and is connected to **insecure communications platforms** such as the Internet.

Remember: File Systems Protection

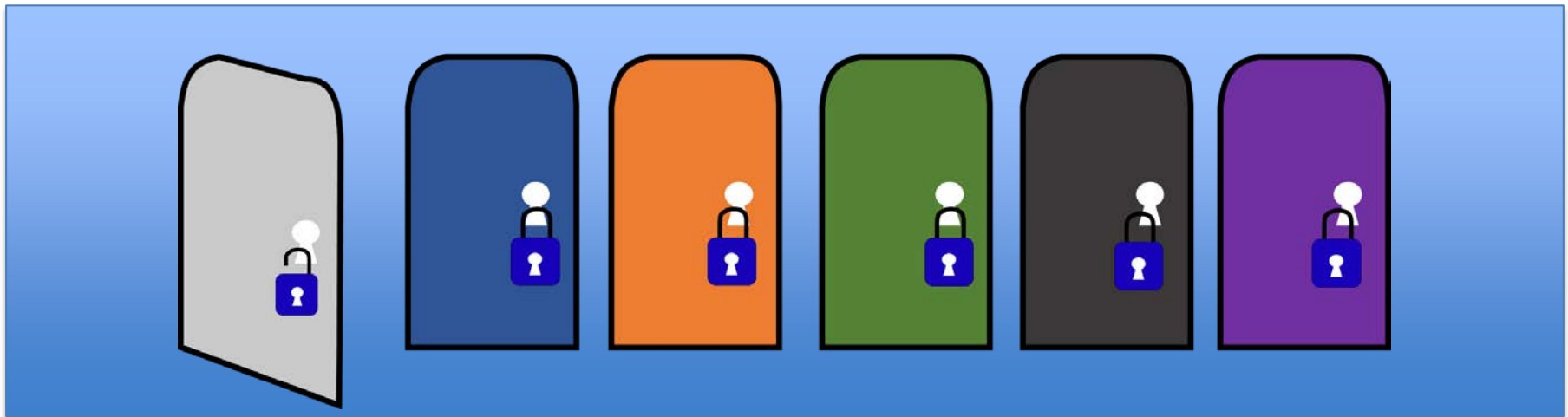
- File owner/creator should be able to control:
 - what can be done
 - by whom

- Types of access include:
 - Read
 - Write
 - Execute
 - [directories] Access children

r	w	x	r	-	x	-	w	x
└──┘			└──┘			└──┘		
user			group			other		
7			5			3		

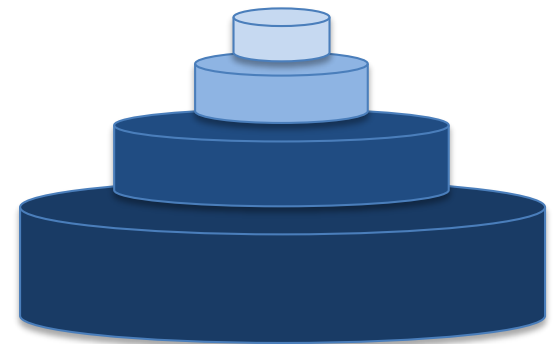
Principles of Protection

- Guiding principle – **principle of least privilege**
 - Programmes, users and systems should be given **just enough privileges** to perform their tasks
 - Limits damage if entity has a bug, gets abused



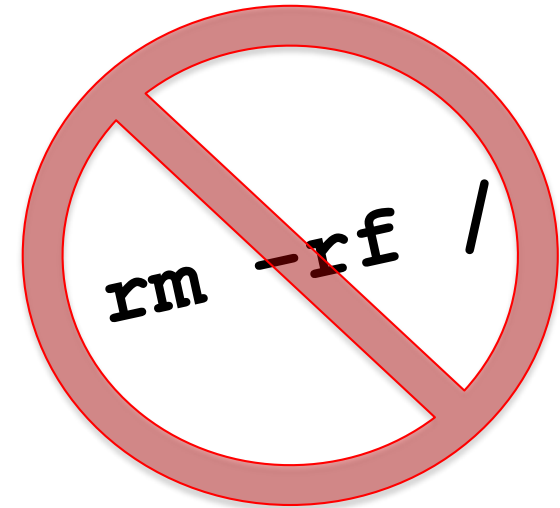
Principles of Protection

- Guiding principle – **principle of least privilege**
 - Can be **static** (during life of system, during life of process) or **dynamic** (changed by process as needed) – domain switching, privilege escalation
 - “Need to know” basis



Least Privilege 1

- Consider one of the tenets of UNIX—that a user should not run as root. Most users innately respect that, fearing an accidental delete operation for which there is no corresponding undelete.
- Because root is virtually omnipotent, the potential for **human error** when a user acts as root is grave, and its consequences far reaching.



Least Privilege 2

- Now consider that rather than human error, damage may result from **malicious attack**. A virus launched by an accidental click on an attachment is one example.
- Another is a **buffer overflow** or other code injection attack that is successfully carried out against a root-privileged process (or, in Windows, a process with administrator privileges).

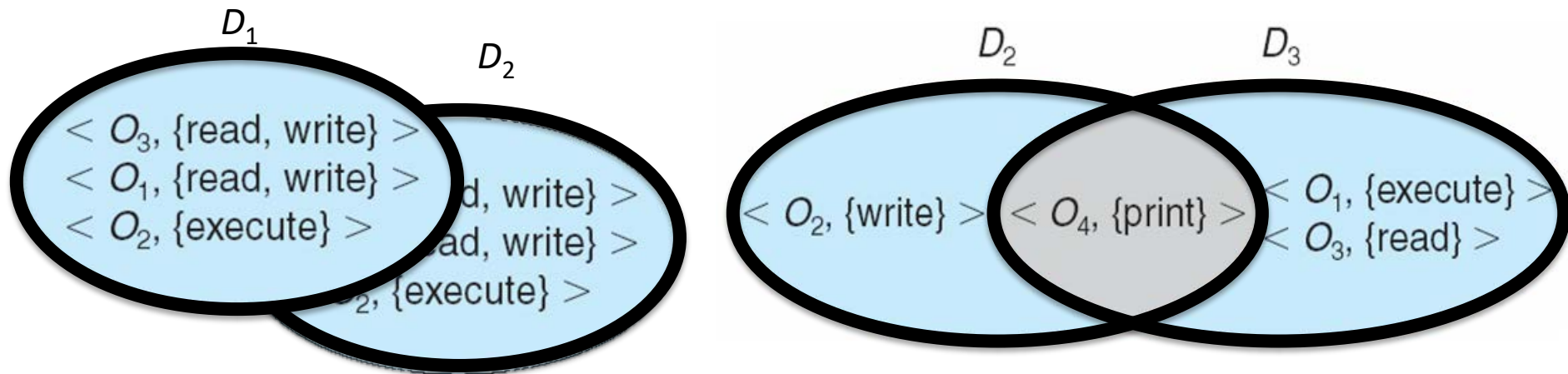
```
=====
You have been hacked. Because you ran
this friendly malware as root all users
have been affected, : - )
=====
```

Systems Protection

- Goals and Principles of Protection
- Protection through Domains
- Access Matrix
- Revocation of Access Rights
- Access Control Policies

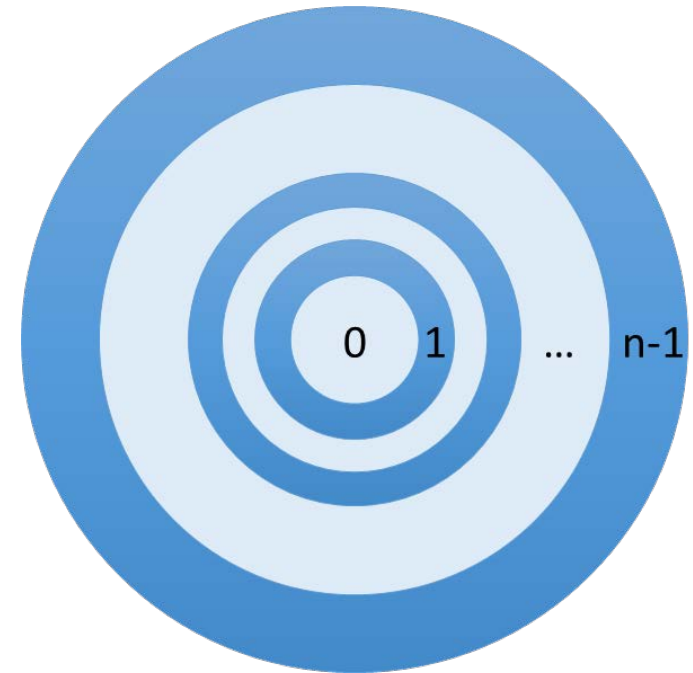
Domain Structure

- Access-right = $\langle \text{object-name, rights-set} \rangle$
where rights-set is a subset of all valid operations that can be performed on the object
- Domain = set of access-rights



Domain Implementation (MULTICS)

- Multics ("Multiplexed Information and Computing Service") was an influential early time-sharing operating system developed by MIT, General Electric and Bell Labs.
- Let D_i and D_j be any two domain rings
- If $j < i \Rightarrow D_i \subseteq D_j$
- A process executing in domain D_j has more privileges than does a process executing in domain D_i



Multics Benefits and Limits

- Ring / hierarchical structure provided more than the basic kernel / user or root / normal user design
- Fairly complex -> more overhead
- But does not allow strict need-to-know
 - Object accessible in D_j but not in D_i , then j must be $< i$
 - But then every segment accessible in D_i also accessible in D_j

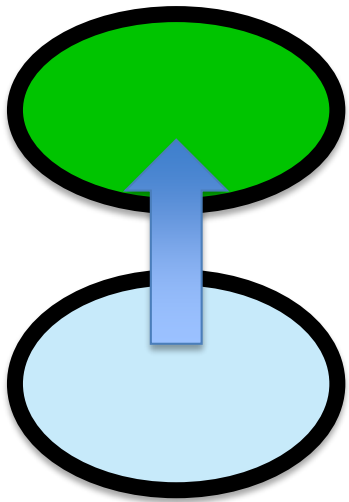
Domain Implementation (UNIX)

- Domain switch accomplished via passwords
 - su command temporarily switches to another user's domain when other domain's password provided
- Domain switching via commands
 - sudo command executes specified command in another domain (if original domain has privilege or password given)

```
$whoami  
troy  
$sudo -u root whoami  
root
```

Domain Implementation (UNIX)

- Each **user-id** names a domain determined by all the file permissions
- Unix allows to switch domains
- Domain Switch accomplished via **file system**



- Each file has associated with it a domain bit (**setuid** bit)
- When file is executed and setuid = on, then user-id is set to owner of the file being executed
- When execution completes user-id is reset

Systems Protection

- Goals and Principles of Protection
- Protection through Domains
- **Access Matrix**
- Revocation of Access Rights
- Access Control Policies

Access Matrix

- View protection as a matrix (access matrix)
- Rows represent domains
- Columns represent objects

Objects domains	Obj1 (file)	Obj2 (file)	Obj2 (file)	Obj2 (disk)	Obj2 (printer)
Domain1	read	read	read	read write execute	read write execute
Domain2	read execute	write	read	read	read
Domain3	read execute	read	read	read	read

Access Matrix

- Access (i, j) is the set of operations that a process executing in Domain (i) can invoke on Object (j)

Objects domains	Obj1 (file)	Obj2 (file)	Obj2 (file)	Obj2 (disk)	Obj2 (printer)
Domain1	read	read	read	read write execute	read write execute
Domain2	read execute	write	read	read	read
Domain3	read execute	read	read	read	read

Groups

- How many groups exist? Well many, depending on the settings of the system. There are several ways to check but one is to look in `/etc/group`

```
troy@bigears:~$ cat /etc/group
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:
tty:x:5:
disk:x:6:
lp:x:7:
mail:x:8:
news:x:9:
uucp:x:10:
man:x:12:
dialout:x:20:
fax:x:21:
...
troy:x:1000:
...
geek:x:1003:zapata,troy,maryam,helen
cobbler:x:1004:zapata,maryam
```

Annotations:

- Name of group (points to `root`)
- x: can be a group password (points to `x`)
- Actual id of the group (points to `0`)
- Member of a group (points to `troy` in the `geek` group line)

Access Matrix

- Different table where owners and groups are different

```
geek:x:1003:zapata,troy,maryam,helen
cobbler:x:1004:zapata,maryam
```

```
-rwxr-xrw- 1 maryam cobbler 0 Mar 27 10:50 goat
-rw--w--w- 1 maryam geek 0 Mar 27 10:44 lion
-rwxrwx-wx 1 zapata geek 0 Mar 27 10:45 penguin
drw-rw--wx 2 zapata cobbler 4096 Mar 27 10:50 reptiles
-rw-rw-r-- 1 troy geek 0 Mar 27 10:44 tiger
```

	goat	lion	penguin	reptiles	tiger
troy	read write	write			
zapata					
maryam					

Access Matrix

- Different table where owners and groups are different

```
geek:x:1003:zapata,troy,maryam,helen
cobbler:x:1004:zapata,maryam
```

```
-rwxr-xrw- 1 maryam cobbler 0 Mar 27 10:50 goat
-rw--w--w- 1 maryam geek 0 Mar 27 10:44 lion
-rwxrwx-wx 1 zapata geek 0 Mar 27 10:45 penguin
drw-rw--wx 2 zapata cobbler 4096 Mar 27 10:50 reptiles
-rw-rw-r-- 1 troy geek 0 Mar 27 10:44 tiger
```

	goat	lion	penguin	reptiles	tiger
troy	read write	write	read write execute	write execute	read write
zapata	read execute	write	read write execute	read write	read write
maryam	read write execute	read write	read write execute	read write	read write

Access Matrix

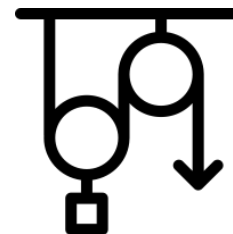
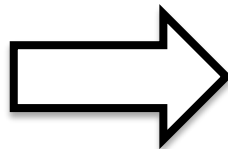
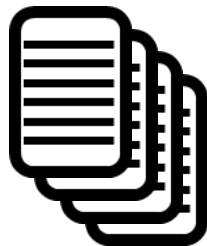
Mechanism v Policy

- Access matrix design separates mechanism from policy
 - Mechanism
 - Operating system provides access-matrix + rules
 - If ensures that the matrix is only manipulated by authorized agents and that rules are strictly enforced
 - Policy
 - User dictates policy
 - Who can access what object and in what mode

Access Matrix

Mechanism v Policy

- The access-matrix scheme provides us with a way to specify a variety of **policies**.
- The **mechanism** enforces the rules specified by the access matrix, ensuring that the semantic properties we have outlined hold.



Access Matrix

Mechanism v Policy

- Specifically: the mechanism **must ensure that a process executing in domain D_i can access only those objects specified in row i** , and then only as allowed by the access-matrix entries.
 - Our choice of **policy** decides **which rights should be included in the $(i,j)th$ entry**.
 - The access matrix **mechanism** enforces those choices.

Systems Protection

- Goals and Principles of Protection
- Protection through Domains
- Access Matrix
- **Revocation of Access Rights**
- Access Control Policies

Revocation of Access Rights

- In a dynamic protection system, we may sometimes need to *revoke* access rights to objects shared by different users.
- Various options to remove the access right of a domain to an object
 - Immediate vs. delayed
 - Selective vs. general
 - Partial vs. total
 - Temporary vs. permanent



Revocation of Access Rights

- **Immediate versus delayed.** Does revocation occur immediately, or is it delayed? If revocation is delayed, can we find out when it will take place?
- **Selective versus general.** When an access right to an object is revoked, does it affect all the users who have an access right to that object, or can we specify a select group of users whose access rights should be revoked?

Revocation of Access Rights

- **Partial versus total.** Can a subset of the rights associated with an object be revoked, or must we revoke all access rights for this object?
- **Temporary versus permanent.** Can access be revoked permanently (that is, the revoked access right will never again be available), or can access be revoked and later be obtained again?

Systems Protection

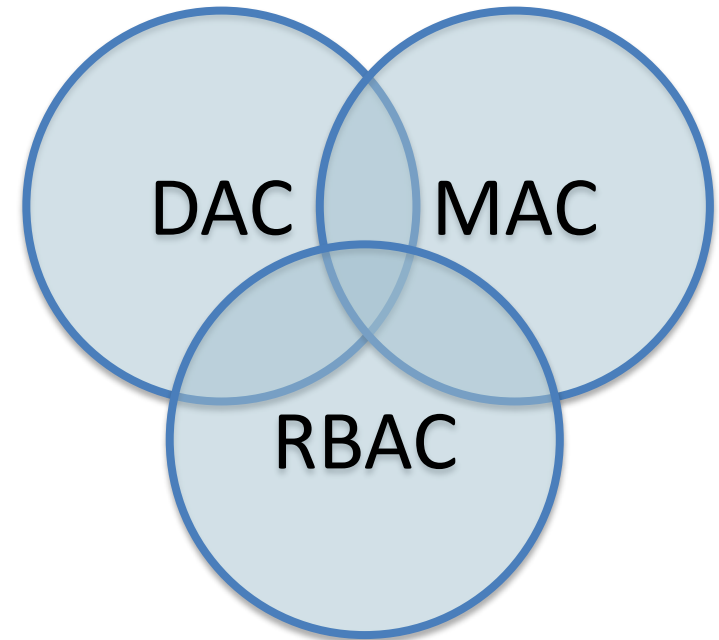
- Goals and Principles of Protection
- Protection through Domains
- Access Matrix
- Revocation of Access Rights
- Access Control Policies

Access Control Policies

- An access control policy dictates what types of access are permitted, under what circumstances, and by whom
- ITU-T Recommendation X.800 defines access control as follows:
 - *The prevention of unauthorized use of a resource, including the prevention of use of a resource in an unauthorized manner.*

Access Control Policies

- Access control policies are generally grouped into the following categories:
 - **Mandatory access control (MAC)**
 - **Discretionary access control (DAC)**
 - **Role-based access control (RBAC)**



Access Control Policies: MAC

- **Mandatory access control (MAC)** Controls access based on comparing security labels with security clearances for objects or users. Policy dictates appropriate mapping between clearances and labels.
- Example:
 - Consider a simple set of labels, ordered according to level of privilege: “**unclassified**,” “**secret**,” and “**top secret**.”
 - A process/user with “**secret**” clearance will be able to create similarly labelled processes, which will then have access to “**unclassified**” and “**secret**” files, but not to “**top secret**” files.

SECRET!!!!

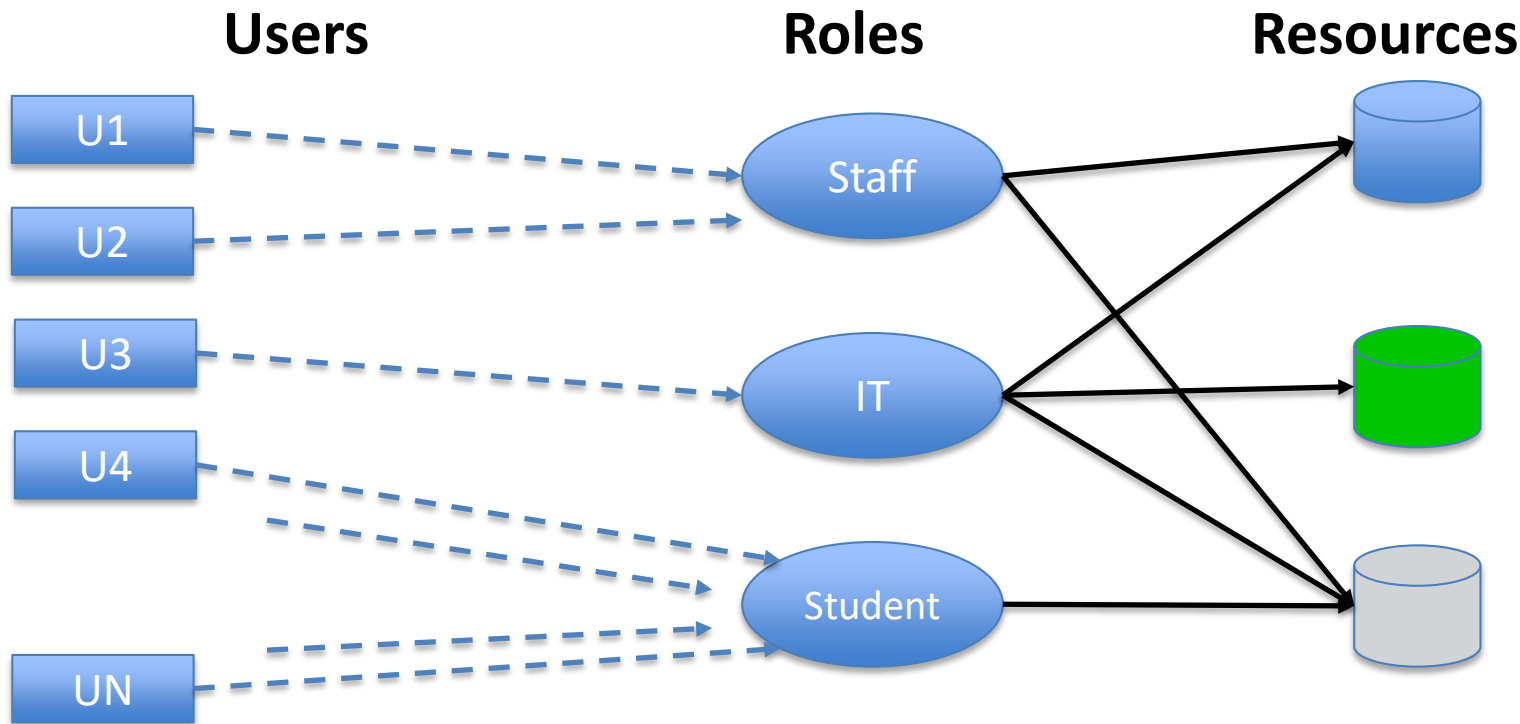
TOP SECRET!!!!

Access Control Policies: DAC

- **Discretionary access control (DAC)** Scheme in which an entity or object (i.e. a user) may enable another entity to access some resource. This policy is termed **discretionary** because an entity might have access rights that permit the entity, by its own volition, to enable another entity to access some resource.
- In **MAC**, it is a centrally controlled, an administrator who changes the levels.
- In contrast in **DAC** is that users have the ability to assign/withhold access rights – to edit cells in the access matrix - for the objects which they own (or for which they have "delegation" rights)

Access Control Policies: RBAC

- **Role-based access control (RBAC)** Controls access based on the roles that users have within the system, and on rules stating what accesses are allowed to users in given roles



Protection $\neq$ Security

- File system protection mechanisms may prevent a process owned by watson from accessing a file owned by sherlock.
- But what if watson can login as sherlock (masquerading)?
 - Then watson can execute processes owned by sherlock
 - So watson can see and modify sherlock's data and processes

Systems Protection

- Goals and Principles of Protection
- Protection through Domains
- Access Matrix
- Revocation of Access Rights
- Access Control Policies